

In-Class Assignment: Build LLM Workflow Selector

Goal

Build an LLM workflow selector that can:

1. Load published workflow definitions from DynamoDB.
 2. Combine workflow definitions into a prompt.
 3. Send the user message and workflow definitions to OpenAI.
 4. Select the best matching workflow.
 5. Return a confidence score.
 6. Extract required fields.
 7. Return missing fields and follow-up questions.
 8. Return whether the request is ready for workflow execution.
-

Step 1: Update requirements.txt

Open:

```
requirements.txt
```

Make sure it includes:

```
fastapi
uvicorn
pydantic
boto3
openai
```

Install dependencies:

```
pip install -r requirements.txt
```

Step 2: Set Environment Variables

Run:

```
export AWS_REGION="us-west-2"
export WORKFLOW_DEFINITIONS_TABLE_NAME="WorkflowDefinitions"
export OPENAI_API_KEY="YOUR_OPENAI_API_KEY"
export OPENAI_MODEL="gpt-5"
```

Do not commit your OpenAI API key to GitHub.

Step 3: Update app/config.py

Open:

```
app/config.py
```

Make sure it contains:

```
import os

AWS_REGION = os.getenv("AWS_REGION", "us-west-2")

WORKFLOW_DEFINITIONS_TABLE_NAME = os.getenv(
    "WORKFLOW_DEFINITIONS_TABLE_NAME",
    "WorkflowDefinitions",
)

OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-5")
```

Step 4: Create Workflow Selector Models

Create:

```
app/models/workflow_selector_models.py
```

Add:

```
from typing import Dict, List, Optional
```

```
from pydantic import BaseModel, Field

class WorkflowSelectionRequest(BaseModel):
    message: str = Field(..., min_length=3)
    user_id: Optional[str] = None
    source: Optional[str] = "unknown"

class WorkflowSelectionResponse(BaseModel):
    selected_workflow_id: Optional[str]
    selected_workflow_type: Optional[str]
    confidence_score: float
    reason: str
    extracted_fields: Dict[str, str]
    missing_fields: List[str]
    missing_field_questions: List[str]
    is_ready_for_workflow: bool
```

Step 5: Check Workflow Definition Service

Open:

```
app/services/workflow_definition_service.py
```

Make sure it has this method:

```
def list_published_workflow_definitions(self):
    return self.workflow_definition_repository.list_published()
```

The selector will use this method to load workflow definitions from DynamoDB.

Step 6: Create LLM Workflow Selector Service

Create:

```
app/services/llm_workflow_selector_service.py
```

Add:

```

import json
from typing import Dict, List, Optional

from openai import OpenAI

from app.config import OPENAI_MODEL
from app.models.workflow_definition_models import WorkflowDefinitionResponse
from app.models.workflow_selector_models import (
    WorkflowSelectionRequest,
    WorkflowSelectionResponse,
)
from app.services.workflow_definition_service import workflow_definition_service

class LLMWorkflowSelectorService:
    def __init__(self):
        self.client = OpenAI()

    def select_workflow(
        self,
        request: WorkflowSelectionRequest,
    ) -> WorkflowSelectionResponse:
        workflow_definitions = (
            workflow_definition_service.list_published_workflow_definitions()
        )

        if not workflow_definitions:
            return WorkflowSelectionResponse(
                selected_workflow_id=None,
                selected_workflow_type=None,
                confidence_score=0.0,
                reason="No published workflow definitions are available.",
                extracted_fields={},
                missing_fields=[],
                missing_field_questions=[],
                is_ready_for_workflow=False,
            )

        raw_selection = self._call_openai(
            user_message=request.message,
            workflow_definitions=workflow_definitions,
        )

        return self._build_response(
            raw_selection=raw_selection,
            workflow_definitions=workflow_definitions,
        )

```

```

def _call_openai(
    self,
    user_message: str,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> Dict:
    system_prompt = """

```

You are a workflow selection engine for a client issue triage platform.

Your job:

1. Read the user's message.
2. Review all available workflow definitions.
3. Select the best matching workflow.
4. Extract required and optional fields from the user message.
5. Identify missing required fields.
6. Return a confidence score between 0 and 1.

Rules:

- Only select a workflow from the provided workflow definitions.
 - If none of the workflows match, selected_workflow_id must be null.
 - Do not invent extracted field values.
 - If a required field is missing, include it in missing_fields.
 - Use the workflow definition's missing_field_question when possible.
 - confidence_score is your estimated confidence in the selected workflow.
- """

```

        workflow_definitions_text = self._format_workflow_definitions(
            workflow_definitions
        )

```

```

        user_prompt = f"""

```

User message:

```
{user_message}
```

Available workflow definitions:

```
{workflow_definitions_text}
```

Return the workflow selection result as JSON.

"""

```

        response = self.client.responses.create(
            model=OPENAI_MODEL,
            input=[
                {
                    "role": "system",
                    "content": system_prompt,
                },
                {

```

```

        "role": "user",
        "content": user_prompt,
    },
],
text={
    "format": {
        "type": "json_schema",
        "name": "workflow_selection_result",
        "strict": True,
        "schema": {
            "type": "object",
            "additionalProperties": False,
            "properties": {
                "selected_workflow_id": {
                    "type": ["string", "null"]
                },
                "selected_workflow_type": {
                    "type": ["string", "null"]
                },
                "confidence_score": {
                    "type": "number"
                },
                "reason": {
                    "type": "string"
                },
                "extracted_fields": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "additionalProperties": False,
                        "properties": {
                            "field_name": {
                                "type": "string"
                            },
                            "field_value": {
                                "type": "string"
                            }
                        },
                        "required": [
                            "field_name",
                            "field_value"
                        ]
                    }
                },
                "missing_fields": {
                    "type": "array",
                    "items": {
                        "type": "string"
                    }
                }
            }
        }
    }
}

```

```

        }
    },
    "missing_field_questions": {
        "type": "array",
        "items": {
            "type": "string"
        }
    }
},
"required": [
    "selected_workflow_id",
    "selected_workflow_type",
    "confidence_score",
    "reason",
    "extracted_fields",
    "missing_fields",
    "missing_field_questions"
]
}
},
)

return json.loads(response.output_text)

def _format_workflow_definitions(
    self,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> str:
    simplified_definitions = []

    for workflow in workflow_definitions:
        simplified_definitions.append(
            {
                "workflow_id": workflow.workflow_id,
                "workflow_type": workflow.workflow_type,
                "name": workflow.name,
                "description": workflow.description,
                "trigger_examples": workflow.trigger_examples,
                "required_fields": [
                    field.model_dump(mode="json")
                    for field in workflow.required_fields
                ],
                "optional_fields": [
                    field.model_dump(mode="json")
                    for field in workflow.optional_fields
                ],
                "min_confidence_score": workflow.min_confidence_score,
            }
        )

```

```

        }
    )

    return json.dumps(simplified_definitions, indent=2)

def _build_response(
    self,
    raw_selection: Dict,
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> WorkflowSelectionResponse:
    selected_workflow_id = raw_selection.get("selected_workflow_id")
    selected_workflow = self._find_workflow_definition(
        selected_workflow_id=selected_workflow_id,
        workflow_definitions=workflow_definitions,
    )

    confidence_score = float(raw_selection.get("confidence_score", 0.0))
    missing_fields = raw_selection.get("missing_fields", [])

    extracted_fields = self._normalize_extracted_fields(
        raw_selection.get("extracted_fields", [])
    )

    is_ready_for_workflow = False

    if selected_workflow is not None:
        is_ready_for_workflow = (
            confidence_score >= selected_workflow.min_confidence_score
            and len(missing_fields) == 0
        )

    return WorkflowSelectionResponse(
        selected_workflow_id=raw_selection.get("selected_workflow_id"),
        selected_workflow_type=raw_selection.get("selected_workflow_type"),
        confidence_score=confidence_score,
        reason=raw_selection.get("reason", ""),
        extracted_fields=extracted_fields,
        missing_fields=missing_fields,
        missing_field_questions=raw_selection.get(
            "missing_field_questions",
            [],
        ),
        is_ready_for_workflow=is_ready_for_workflow,
    )

def _normalize_extracted_fields(self, raw_fields: List[Dict]) -> Dict[str,
str]:
    extracted_fields = {}

```

```

    for field in raw_fields:
        field_name = field.get("field_name")
        field_value = field.get("field_value")

        if field_name and field_value:
            extracted_fields[field_name] = field_value

    return extracted_fields

def _find_workflow_definition(
    self,
    selected_workflow_id: Optional[str],
    workflow_definitions: List[WorkflowDefinitionResponse],
) -> Optional[WorkflowDefinitionResponse]:
    if selected_workflow_id is None:
        return None

    for workflow in workflow_definitions:
        if workflow.workflow_id == selected_workflow_id:
            return workflow

    return None

```

```
llm_workflow_selector_service = LLMWorkflowSelectorService()
```

Step 7: Create Workflow Selector Router

Create:

```
app/routers/workflow_selector_router.py
```

Add:

```

from fastapi import APIRouter

from app.models.workflow_selector_models import (
    WorkflowSelectionRequest,
    WorkflowSelectionResponse,
)
from app.services.llm_workflow_selector_service import (
    llm_workflow_selector_service,

```

```

)

router = APIRouter(
    prefix="/workflow-selector",
    tags=["Workflow Selector"],
)

@router.post(
    "/select",
    response_model=WorkflowSelectionResponse,
)
def select_workflow(request: WorkflowSelectionRequest):
    return llm_workflow_selector_service.select_workflow(request)

```

Step 8: Update app/main.py

Open:

```
app/main.py
```

Make sure it includes the workflow selector router:

```

from fastapi import FastAPI

from app.routers.ticket_router import router as ticket_router
from app.routers.workflow_definition_router import (
    router as workflow_definition_router,
)
from app.routers.workflow_selector_router import (
    router as workflow_selector_router,
)

app = FastAPI(
    title="AI Ticket Workflow Platform",
    description="A backend platform for client issue triage and workflow automation.",
    version="0.3.0",
)

```

```
app.include_router(ticket_router)
app.include_router(workflow_definition_router)
app.include_router(workflow_selector_router)

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

Step 9: Start Server

Run:

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Step 10: Test Published Workflow Definitions

Call:

```
GET /workflow-definitions/published
```

Expected result:

```
Returns the published workflow definitions created earlier.
```

Make sure the first 4 workflow definitions exist before testing the selector.

Step 11: Test Selector — Stock Data Issue

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "TSLA price has not updated for 30 minutes.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_stock_data_issue_v1",
  "selected_workflow_type": "STOCK_DATA_ISSUE",
  "confidence_score": 0.8,
  "reason": "...",
  "extracted_fields": {
    "symbol": "TSLA",
    "issue_description": "...",
    "time_window": "30 minutes"
  },
  "missing_fields": [],
  "missing_field_questions": [],
  "is_ready_for_workflow": true
}
```

Step 12: Test Selector — Watchlist Bug

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "My watchlist disappeared this morning.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_watchlist_bug_v1",
  "selected_workflow_type": "WATCHLIST_BUG",
  "confidence_score": 0.8,
  "reason": "...",
  "extracted_fields": {
    "watchlist_issue": "watchlist disappeared this morning"
  },
  "missing_fields": [],
  "missing_field_questions": [],
  "is_ready_for_workflow": true
}
```

Step 13: Test Selector — Price Alert Bug

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "I did not get an alert when TSLA dropped below 200.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_price_alert_bug_v1",
```

```
"selected_workflow_type": "PRICE_ALERT_BUG",
"confidence_score": 0.8,
"reason": "...",
"extracted_fields": {
  "alert_issue": "...",
  "symbol": "TSLA",
  "target_price": "200"
},
"missing_fields": [],
"missing_field_questions": [],
"is_ready_for_workflow": true
}
```

Step 14: Test Selector — General Product Question

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "Where can I see after-hours stock prices?",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_general_product_question_v1",
  "selected_workflow_type": "GENERAL_PRODUCT_QUESTION",
  "confidence_score": 0.8,
  "reason": "...",
  "extracted_fields": {
    "question": "Where can I see after-hours stock prices?"
  },
  "missing_fields": [],
  "missing_field_questions": [],
}
```

```
"is_ready_for_workflow": true
}
```

Step 15: Test Selector — Missing Required Field

Call:

```
POST /workflow-selector/select
```

Request body:

```
{
  "message": "The stock price is wrong.",
  "user_id": "user_123",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "selected_workflow_id": "wf_stock_data_issue_v1",
  "selected_workflow_type": "STOCK_DATA_ISSUE",
  "confidence_score": 0.7,
  "reason": "...",
  "extracted_fields": {
    "issue_description": "stock price is wrong"
  },
  "missing_fields": [
    "symbol"
  ],
  "missing_field_questions": [
    "Which stock symbol has the issue?"
  ],
  "is_ready_for_workflow": false
}
```

Step 16: Commit Code

Run:

```
git status
git add .
git commit -m "Add LLM workflow selector"
git push origin main
```

Acceptance Criteria

The assignment is complete when:

1. `POST /workflow-selector/select` loads published workflow definitions from DynamoDB.
2. The selector calls OpenAI with the user message and workflow definitions.
3. The selector returns `selected_workflow_id`.
4. The selector returns `selected_workflow_type`.
5. The selector returns `confidence_score`.
6. The selector returns `reason`.
7. The selector returns `extracted_fields`.
8. The selector returns `missing_fields`.
9. The selector returns `missing_field_questions`.
10. `is_ready_for_workflow` is `true` when confidence is high enough and all required fields are extracted.
11. `is_ready_for_workflow` is `false` when required fields are missing.

Questions

1. Why does the selector load workflow definitions from DynamoDB?
2. Why does the selector send all published definitions to OpenAI?
3. What does `confidence_score` mean?
4. What does `is_ready_for_workflow` mean?
5. Why does the backend compute `is_ready_for_workflow` instead of fully trusting the LLM?
6. Why should the LLM return missing fields?
7. Why should the LLM not invent extracted field values?
8. How can we add a new workflow type without changing selector code?